Algoritmo e Laboratório de Programação II

Aula 1 – Matrizes – Parte 1 (Nível Avançado)

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

Introdução

- Matriz é uma coleção de variáveis do mesmo tipo que compartilham um mesmo nome.
- Imagine o seguinte problema:
- Calcular a média aritmética das notas de prova de cinco alunos.
- Você poderia escrever o seguinte programa:

Introdução

Prog1.c

```c
/* Notas.C */
/* Calcula a média de 5 notas (não usa matriz) */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int notas0, nota1, nota2, nota3, nota4;
    float media = 0.0;

    printf("Digite a nota do aluno 1 ");
    printf("Digite a nota do aluno 2 ");
    printf("Digite a nota do aluno 3 ");
    printf("Digite a nota do aluno 4 ");
    printf("Digite a nota do aluno 5 ");

    media =(nota0 + nota1 + nota2 + nota3 + nota4) / 5.0;
    printf("Média das notas: %.2f\n", media);

    system("PAUSE");
    return 0;
}
```

Introdução

- Imagine agora se você pretendesse encontrar a média aritmética das notas de uma classe com 50 alunos, ou da escola toda, com 2 mil alunos?
- Seria uma tarefa bastante volumosa!
- É evidente que precisamos de uma maneira conveniente para referenciar tais coleções de dados similares.

Introdução

- Matriz é o tipo de dado oferecido por C para esse propósito, pois é um conjunto de variáveis de mesmo tipo e referenciadas por um único nome.
- Cada variável é diferenciada por meio de um número chamado índice.
- Os colchetes são usados para conter o índice.

Introdução

Prog2.c

```c
/* Notas.C */
/* Calcula a média de 5 notas (usa matriz) */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int notas[5];
    int i;
    float media = 0.0;

    for( i=0 ; i < 5 ; i++ )
    {
        printf("Digite a nota do aluno %d ", i+1);
        scanf("%d",&notas[i]);
        media += notas[i];
    }

    media /= 5.0;
    printf("Média das notas: %.2f\n", media);

    system("PAUSE");
    return 0;
}
```

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

Declaração da Matriz

- Em C, matrizes precisam ser declaradas como quaisquer outras variáveis, para que o compilador conheça o tipo de seus elementos e reserve espaço de memória suficiente para armazená-las.
- Os elementos da matriz são guardados numa sequência contínua de memória, isto é, um seguido ao outro, o que não ocorre quando criamos variáveis separadas.

Declaração da Matriz

- O que diferencia a declaração de uma matriz de qualquer outra variável é a parte que acompanha seu nome, isto é, o par de colchetes ([e]) que envolve um número inteiro, indicando o tamanho da matriz.
- A instrução

```
8        int notas[5];
```

informa que a matriz notas é formada por cinco elementos do tipo int.

Declaração da Matriz

- Por definição, uma matriz é composta por elementos de um único tipo.
- O valor que dimensiona a matriz, na sua declaração, deve ser uma constante inteira.
- Assim, não podemos dimensionar uma matriz por meio de uma variável.

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

Referenciação dos Elementos da Matriz

- Cada um dos elementos da matriz é referenciado individualmente por meio de um número inteiro, entre colchetes, após o nome da matriz.
- Quando referencia um elemento, esse número tem um significado diferente daquele da declaração da matriz, sua posição é especificada por esse número.

Referenciação dos Elementos da Matriz

- Os elementos são sempre numerados por índices iniciados por zero.
- Por exemplo, a instrução:

```
notas[2] = 90;
```

- Atribui o valor 90 ao terceiro elemento da matriz, pois a numeração começa em zero.
- O último elemento possui um índice de uma unidade menor que a dimensão da matriz.

Referenciação dos Elementos da Matriz

- O índice utilizado para referenciar elementos de uma matriz pode ser o valor de uma variável inteira ou uma constante.
- Em nosso programa, utilizamos a variável i como índice.

Referenciação dos Elementos da Matriz

- Observe a instrução:

```
scanf("%d", &notas[i]);
```

- Quando escrevemos notas[i], estamos escrevendo o nome de uma variável do tipo int como outra qualquer.
- Assim, em todo lugar em que podemos utilizar o nome de uma variável int, podemos usar notas[i].

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

Matrizes de Outros Tipos de Elementos

- O fato de uma matriz ser composta por uma série de elementos de um único tipo permite a escolha de qualquer tipo de variável para ela.
- Suponhamos que você queira que as notas dos seus alunos fiquem no intervalo de 0 a 10 e que seja desprezada a parte fracionária.
- A solução seria usar uma matriz de elementos do tipo float.

Matrizes de Outros Tipos de Elementos

Prog3.c

```c
/* NotasF.C */
/* Calcula a média das notas de 5 alunos usando matriz float */
#include <stdio.h>
#include <stdlib.h>

#define TAMANHO 5 /* não podemos usar const */

int main()
{

    float notas[TAMANHO] , media=0.0;
    int i;

    for( i=0 ; i < TAMANHO ; i++ )
    {
        printf("Digite a nota do aluno %d ", i+1);
        scanf("%f",&notas[i]);
        media += notas[i];
    }
    media /= 5.0;
    printf("Média das notas: %.2f\n", media);

    system("PAUSE");
    return 0;
}
```

Matrizes de Outros Tipos de Elementos

- O programa apresenta um novo detalhe: o uso da diretiva #define para criar a constante que dimensiona a matriz e controla o limite do laço:

```
6    #define TAMANHO 5 /* não podemos usar const */
```

- Usar # define no lugar de um número constante facilita a alteração do tamanho da matriz.

Matrizes de Outros Tipos de Elementos

- Se você desejar calcular a média das notas de mais alunos, somente uma linha do programa deve ser alterada.
- Por exemplo, para calcular a média das notas de 30 alunos, devemos alterar somente a instrução de declaração da constante:

```
#define TAMANHO 30
```

Matrizes de Outros Tipos de Elementos

- Um Número Desconhecido de Elementos:
  - Nos exemplos anteriores, utilizamos um número fixo de notas.
  - Como faríamos se não conhecêssemos de antemão quantos itens entrariam na matriz?
  - O programa seguinte aceita até 50 notas e pode ser facilmente modificado para aceitar qualquer número de notas:

Matrizes de Outros Tipos de Elementos

Prog4.c

```
1   /* NotasF1.C */
2   /* Calcula a média das notas de qualquer número de alunos */
3   #include <stdio.h>
4   #include <stdlib.h>
5
6   #define TAMANHO 50
7
8   int main()
9   {
10
11      float notas[TAMANHO] , media=0.0;
12      int i=0,j;
13
14      do
15      {
16          printf("Digite a nota do aluno %d ", i+1);
```

Matrizes de Outros Tipos de Elementos

```
16          printf("Digite a nota do aluno %d ", i+1);
17          scanf("%f",&notas[i]);
18      }
19      while( notas[i++] >= 0.0);
20
21      i--; /* remove o item de término */
22
23      for(j=0 ; j < i ; j++)
24          media += notas[j];
25
26      media /= i;
27      printf("Média das notas: %.2f\n", media);
28
29      system("PAUSE");
30      return 0;
31  }
```

Matrizes de Outros Tipos de Elementos

- Um Número Desconhecido de Elementos:
  - O laço for foi substituído pelo laço do-while.
  - Esse laço repete a solicitação da nota do usuário, armazenando-a na matriz notas[], até que seja fornecido um valor menor que zero.
  - Quando o último item for digitado, a variável i terá alcançado um valor acima do total de itens.
  - Isso é verdadeiro, pois é contado o número negativo de finalização da entrada. Assim, devemos subtrair 1 do valor contido em i para encontrar o número correto de intens.

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

Verificação de Limites

- A nossa matriz foi dimensionada em 50 e este número permite a entrada de até 50 notas.
- Suponhamos que o usuário decida inserir mais de 50 notas.
- Como expulsar os dados excedentes?
- A linguagem C não realiza verificação de limite em matrizes, por isso nada impede que você vá além do fim de uma matriz.

Verificação de Limites

- Se você transpuser o fim da matriz durante uma operação de atribuição, os valores sobressalentes irão se sobrepor a outros dados da memória.
- Estes valores serão armazenados em sequência, seguindo os elementos da matriz na memória.

Verificação de Limites

- Como não foi reservado espaço para guardá-los, eles irão se sobrepor a outras variáveis ou até mesmo a uma parte do código do próprio programa que por acaso esteja nesta memória
- Isso acarretará resultados imprevisíveis, e nenhuma mensagem de erro do compilador avisará o que está ocorrendo.

Verificação de Limites

- Em linguagem C, não há sistema que avise quando o limite de uma matriz for excedido.
- Providenciar a verificação do limite de uma matriz é responsabilidade do programador.
- A solução é não permitir que o usuário digite dados acima do limite em elementos da matriz.
- Para isso, modificaremos o laço do-while, do programa anterior, como segue:

Verificação de Limites

Prog5.c

```
1  /* NotasF1.C */
2  /* Calcula a média das notas de qualquer número de alunos */
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  #define TAMANHO 50
7
8  int main()
9  {
10
11     float notas[TAMANHO] , media=0.0;
12     int i=0,j;
13
14     do
15     {
16         if(i >= TAMANHO)
17         {
18             printf("BUFFER LOTADO.\n");
19             i++;
20             break;
21         }
```

Verificação de Limites

```
21          }
22          printf("Digite a nota do aluno %d ", i+1);
23          scanf("%f",&notas[i]);
24      }
25      while( notas[i++] >= 0.0);
26
27      i--; /* remove o item de término */
28
29      for(j=0 ; j < i ; j++)
30          media += notas[j];
31
32      media /= i;
33      printf("Média das notas: %.2f\n", media);
34
35      system("PAUSE");
36      return 0;
37  }
```

Verificação de Limites

- Com essa modificação, quando i atingir 50, que é um acima do último índice da matriz, a mensagem “BUFFER LOTADO.” será impressa e o comando break fará com que o controle saia do laço do-while e passe para a segunda parte do programa.

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

Inicialização de Matrizes

- Você pode fornecer valores a cada elemento da matriz na mesma instrução de sua declaração.
- Veja um exemplo que calcula o número de dias transcorridos do início do ano até a data especificada pelo usuário.
- O programa verifica se o ano é ou não bissexto.

Inicialização de Matrizes

Prog6.c

```c
/* NumDias.C */
/* Imprime o número de dias do ano até a data especificada */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int dmes[12]= { 31,28,31,30,31,30,31,31,30,31,30,31};

    int dia, mes, ano, total, i;

    printf("Digite a data no formato DD/MM/AAAA: ");

    /* %*c lê caractere do teclado e não atribui a nada */
    scanf("%d%*c%d%*c%d", &dia, &mes, &ano);

    if( ((!(ano%4) && ano%100)) || !(ano%400))
        dmes[1]=29;      /* Ano bissexto */

    total=dia;

    for(i=0; i < mes-1; i++)
        total+=dmes[i];

    printf("Total de dias transcorridos desde o início do ano: %d\n",
           total);

    system("PAUSE");
    return 0;
}
```

Inicialização de Matrizes

- O programa inicializa a matriz dmes[] na instrução:

```
8       int dmes[12]= { 31,28,31,30,31,30,31,31,30,31,30,31};
```

- A lista de valores é colocada entre chaves e os valores são separados por vírgulas.
- Eles são atribuídos na sequência em que são escritos, isto é, dmes[0] terá o valor inicial 31 dmes[1] terá o valor inicial 28 e assim por diante.

Inicialização de Matrizes

- Observe que há um ponto-e-virgula após o fechamento das chaves, na inicialização da matriz.
- Em C, as estruturas de programação como laços, funções e comandos de decisão não admitem um ponto-e-virgula após a chave de seus blocos de código, enquanto as estruturas de dados sempre terminam por ponto-e-virgula após a chave de fechamento.

Inicialização de Matrizes

- Matrizes só podem ser inicializadas se forem criadas em tempo de compilação.
- Assim, para inicializar uma matriz, ela deve ser da classe static ou extern.
- Se uma matriz for criada dentro de uma função e inicializada sem que seja escrito explicitamente static, o compilador se encarregará de criá-la como static ou apresentará erro.

Inicialização de Matrizes

```
static int dmes[12]= { 31,28,31,30,31,30,31,31,30,31,30,31};
```

- A instrução de declaração de uma matriz inicializada pode suprimir a dimensão da matriz, restando apenas um par de colchetes vazios:

```
int dmes[ ]= { 31,28,31,30,31,30,31,31,30,31,30,31};
```

- Se nenhum número for dado para dimensionar a matriz, o compilador contará o número de valores inicializados e o fixará como dimensão da matriz.

Inicialização de Matrizes

- Caso você especifique a dimensão da matriz e use menos inicializadores que esse tamanho, os elementos não inicializados conterão zeros.
- Se for colocado um número de inicializadores maior que o necessário, o compilador emitirá um erro.
- Lamentavelmente, em C, não há como especificar a repetição de um inicializador nem inicializar um elemento no meio de uma matriz sem o fazer com todos os anteriores.

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

Matrizes de Mais de Uma Dimensão

- Os elementos de uma matriz podem ser de qualquer tipo, incluindo matrizes.
- Os exemplos vistos até agora são de matrizes de uma dimensão, isto é, matrizes nas quais os elementos são de tipos simples.
- Diremos que uma matriz é de duas dimensões quando seus elementos forem matrizes de uma dimensão.

Matrizes de Mais de Uma Dimensão

- Na verdade, em C, o termo duas dimensões não faz sentido, pois todas as matrizes são armazenadas na memória de forma linear.
- Usaremos esse termo para indicar matrizes em que os elementos são outras matrizes.
- Com dois pares de colchetes, obtemos uma matriz de duas dimensões, e com cada par de colchetes adicionais obtemos matrizes com uma dimensão a mais.

Matrizes de Mais de Uma Dimensão

- Para ilustrar esse conceito, apresentaremos um programa que usa uma matriz de duas dimensões para criar 50 combinações para a loteria (sena).

Matrizes de Mais de Uma Dimensão

Prog7.c

```c
/* Loteria.C */
/* Imprime 50 combinações de jogos para a Sena */
#include <stdio.h>
#include <stdlib.h>
#define JOGOS 50
#define N 6

int main()
{
    int matriz[JOGOS][N], k, j;

    for(k = 0; k < JOGOS; k++)
        for(j=0 ; j < N; j++)
            matriz[k][j]=rand()%60+1;/*gera núm. aleatório de 1 a 60*/

    for(k = 0; k < JOGOS; k++)
    {
        printf("Combinacao %2d:    ", k+1);
        for(j = 0; j < N; j++)/* Imprime as dezenas na tela*/
            printf("%2d   ",matriz[k][j]);
        printf("\n");
    }
    system("PAUSE");
    return 0;
}
```

Matrizes de Mais de Uma Dimensão

- O próximo exemplo usa uma matriz de duas dimensões para criar uma grade do jogo-da-velha.
- Inicialmente, a matriz é preenchida com pontos.
- Depois, o programa forma um ciclo por meio de um laço while no qual a matriz é impressa, verifica se algum jogador já ganhou ou se houve empate, solicita ao jogador que digite um par de coordenadas e finalmente atribui o caractere ‘o’ ou ‘x’ ao elemento da matriz correspondente às coordenadas entradas.

Matrizes de Mais de Uma Dimensão

- Executando esse programa, você verá como trabalha um sistema de coordenadas de duas dimensões.
- Digite pares de números como (0 0), (1 2), (2 2) etc.
- A coordenada horizontal, ou seja, o número da linha, é fornecida primeiro.
- Não esqueça de que os índices começam em zero.

Matrizes de Mais de Uma Dimensão

Prog8.c

```c
/* JogoDaVelha.C */
/* Jogo-da-velha. Mostra matriz de duas dimensões */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    unsigned char m[3][3]; /* Matriz de duas dimensões */
    int lin, col, j=0;

    const int TRUE = 1;
    const char O='o', X ='x';

    printf("Digite coordenadas na forma lin col :\n");

    for(lin=0; lin < 3 ; lin++)
        for(col=0; col < 3; col++)
            m[lin][col]='.';

    while(TRUE)
    {
        for(lin=0; lin < 3 ; lin++)
        {
            for(col=0; col < 3; col++)
                printf("%c ", m[lin][col]);
            printf("\n");
        }
```

Matrizes de Mais de Uma Dimensão

```
29          /* Verifica se primeiro jogador ganhou */
30          if((m[0][0]==O && m[0][1]==O && m[0][2]==O)||
31                  (m[1][0]==O && m[1][1]==O && m[1][2]==O)||
32                  (m[2][0]==O && m[2][1]==O && m[2][2]==O)||
33                  (m[0][0]==O && m[1][0]==O && m[2][0]==O)||
34                  (m[0][1]==O && m[1][1]==O && m[2][1]==O)||
35                  (m[0][2]==O && m[1][2]==O && m[2][2]==O)||
36                  (m[0][0]==O && m[1][1]==O && m[2][2]==O)||
37                  (m[0][2]==O && m[1][1]==O && m[2][0]==O))
38          {
39              printf("\aVoce ganhou, primeiro jogador!!!\n");
40              break;
41          }
42
43          /* Verifica se segundo jogador ganhou */
44          if((m[0][0]==X && m[0][1]==X && m[0][2]==X)||
45                  (m[1][0]==X && m[1][1]==X && m[1][2]==X)||
46                  (m[2][0]==X && m[2][1]==X && m[2][2]==X)||
47                  (m[0][0]==X && m[1][0]==X && m[2][0]==X)||
48                  (m[0][1]==X && m[1][1]==X && m[2][1]==X)||
49                  (m[0][2]==X && m[1][2]==X && m[2][2]==X)||
50                  (m[0][0]==X && m[1][1]==X && m[2][2]==X)||
51                  (m[0][2]==X && m[1][1]==X && m[2][0]==X))
52          {
53              printf("\aVoce ganhou, segundo jogador!!!\n");
54              break;
55          }
```

Matrizes de Mais de Uma Dimensão

```
57          if(j==9)
58          {
59              printf("\aEmpatou!!!\n");
60              break;
61          }
62
63          printf("Coordenadas: ");
64          scanf("%d%d", &lin, &col);
65
66          if(m[lin][col] == '.') /* Casa livre ? */
67          {
68              if(j%2) m[lin][col]=X;
69              else m[lin][col]=O;
70              j++;
71          }
72      }
73
74      system("PAUSE");
75      return 0;
76  }
```

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

Inicialização de Matrizes de Duas Dimensões

- As matrizes de duas dimensões são inicializadas da mesma maneira que as de dimensão única, isto é, os elementos são colocados entre chave depois do sinal de igual e separados por vírgulas.
- Como cada elemento de uma matiz de duas dimensões é por sua vez uma matriz, então cada elemento deve estar entre chaves e os elementos internos devem ser separados por vírgulas.

Inicialização de Matrizes de Duas Dimensões

- Como exemplo, vamos reescrever o programa do jogo da velha inicializando a matriz com pontos.

Inicialização de Matrizes de Duas Dimensões

Prog9.c

```c
/* JogoDaVelha1.C */
/* Jogo-da-velha. Mostra matriz de duas dimensões inicializada */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    const char pt='.';
    unsigned char m[3][3]= /* Matriz de duas dimensões inicializada */
    {{pt, pt, pt}, {pt, pt, pt}, {pt, pt, pt}};

    int lin, col, j=0;

    const int TRUE = 1;
    const char O='o', X ='x';

    printf("Digite coordenadas na forma lin col :\n");

    while(TRUE)
    {
        for(lin=0; lin < 3 ; lin++)
        {
            for(col=0; col < 3; col++)
                printf("%c ", m[lin][col]);
            printf("\n");
        }
```

Inicialização de Matrizes de Duas Dimensões

```
28      /* Verifica se primeiro jogador ganhou */
29      if((m[0][0]==O && m[0][1]==O && m[0][2]==O)||
30              (m[1][0]==O && m[1][1]==O && m[1][2]==O)||
31              (m[2][0]==O && m[2][1]==O && m[2][2]==O)||
32              (m[0][0]==O && m[1][0]==O && m[2][0]==O)||
33              (m[0][1]==O && m[1][1]==O && m[2][1]==O)||
34              (m[0][2]==O && m[1][2]==O && m[2][2]==O)||
35              (m[0][0]==O && m[1][1]==O && m[2][2]==O)||
36              (m[0][2]==O && m[1][1]==O && m[2][0]==O))
37      {
38          printf("\aVoce ganhou, primeiro jogador!!!\n");
39          break;
40      }
41
42      /* Verifica se segundo jogador ganhou */
43      if((m[0][0]==X && m[0][1]==X && m[0][2]==X)||
44              (m[1][0]==X && m[1][1]==X && m[1][2]==X)||
45              (m[2][0]==X && m[2][1]==X && m[2][2]==X)||
46              (m[0][0]==X && m[1][0]==X && m[2][0]==X)||
47              (m[0][1]==X && m[1][1]==X && m[2][1]==X)||
48              (m[0][2]==X && m[1][2]==X && m[2][2]==X)||
49              (m[0][0]==X && m[1][1]==X && m[2][2]==X)||
50              (m[0][2]==X && m[1][1]==X && m[2][0]==X))
51      {
52          printf("\aVoce ganhou, segundo jogador!!!\n");
53          break;
54      }
```

Inicialização de Matrizes de Duas Dimensões

```
56          if(j==9)
57          {
58              printf("\aEmpatou!!!\n");
59              break;
60          }
61
62          printf("Coordenadas: ");
63          scanf("%d%d", &lin, &col);
64
65          if(m[lin][col] == '.') /* Casa livre ? */
66          {
67              if(j%2) m[lin][col]=X;
68              else m[lin][col]=O;
69              j++;
70          }
71      }
72
73      system("PAUSE");
74      return 0;
75  }
```

Inicialização de Matrizes de Duas Dimensões

- A matriz m[] tem três elementos: m[0], m[1] e m[2].
- Cada elemento me m[] é, por sua vez, uma matriz de três elementos do tipo char.
- Assim, m[0], m[1] e m[2] são nomes de matrizes de elementos do tipo char.

Inicialização de Matrizes de Duas Dimensões

- Para inicializar uma matriz, devemos colocar a lista de elementos em ordem, entre chaves e separados por vírgulas, exatamente como fizemos ao inicializar uma matriz de uma dimensão:

```
const char pt='.';
unsigned char m[3][3]= /* Matriz de duas dimensões inicializada */
{{pt, pt, pt}, {pt, pt, pt}, {pt, pt, pt}};
```

- A lista de valores usada para inicializar a matriz apresenta constantes separadas por vírgulas e envoltas por chaves.

Inicialização de Matrizes de Três Dimensões

- Uma matriz de três dimensões é aquela em que cada elemento é uma matriz de duas dimensões.
- Observe o fragmento de programa a seguir que inicializa uma matriz de três dimensões:

Inicialização de Matrizes de Três Dimensões

```
int tresd[3][2][4] =
{
    {
        {1, 2, 3. 4},
        {5, 6, 7, 8}
    },
    {

        {7, 9, 3, 2},
        {4, 6, 8, 3}
    },
    {
        {7, 2, 6, 3},
        {0, 1, 9, 4}
    },
};
```

Inicialização de Matrizes de Três Dimensões

- A matriz tresd[] tem três elementos: tresd[0], tresd[1] e tresd[2].
- Cada um de seus elementos é uma matriz de duas dimensões.
- O primeiro índice é [2], pois o zero pertence ao terceiro grupo.
- O segundo índice é [1], pois o zero está na segunda matriz do terceiro grupo.
- O terceiro índice é [0], pois é o primeiro dos quatro números.

Inicialização de Matrizes de Três Dimensões

- Então:

```
tresd[2][1][0] == 0
```

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

Matrizes como argumentos de Funções

- Temos visto exemplos da passagem de vários tipos de variáveis como argumentos de funções.
- As matrizes também podem ser passadas como argumento para uma função.
- O programa seguinte é uma variação do programa que faz a média das notas, que usa uma função para calcular a média aritmética dos valores de uma matriz.

Matrizes como argumentos de Funções

Prog10.c

```c
/* Prog10.C */
/* Mostra a passagem de matrizes para funções como argumento */
#include <stdio.h>
#include <stdlib.h>
#define TAMANHO 50

float media(float[], int); /*Protótipo*/

int main()
{
    float notas[TAMANHO], m;
    int i=0;

    do
    {
        printf("Digite a nota do aluno %d ", i+1);
        scanf("%f",&notas[i]);
    }
    while( notas[i++] >= 0.0);

    i--; /* remove o item de término */
```

Matrizes como argumentos de Funções

```
23      m = media(notas, i);
24
25      printf("Média das notas: %.2f\n", media);
26
27      system("PAUSE");
28      return 0;
29  }
30
31  float media(float lista[], int tamanho)
32  {
33      int i;
34      float m = 0.0;
35      for(i=0 ; i < tamanho ; i++)m += lista[i];
36      return m/tamanho;
37  }
```

Matrizes como argumentos de Funções

- A primeira parte deste programa lê a lista das notas enquanto não for digitada uma nota negativa.
- O único elemento novo deste programa é a instrução:

```
23          m = media(notas, i);
```

- Ela é uma chamada à função media(), que retorna a média das notas armazenadas na matriz.

Matrizes como argumentos de Funções

- A parte interessante a ser considerada aqui e como foi passada a matriz para a função: usamos unicamente o nome da matriz.
- Para referenciar um elemento da matriz já sabemos que a sintaxe é notas[i]; mas o que representa o nome da matriz sem colchetes?
- O nome de uma matriz, desacompanhado de colchetes, representa o endereço de memória em que a matriz foi armazenada.

Matrizes como argumentos de Funções

- O nome de uma variável simples representa o valor nela contido; entretanto, o nome de uma matriz representa seu endereço de memória.
- Esse endereço é o do primeiro elemento da matriz.
- Em outras palavras, notas é o endereço de notas[0], que é uma variável do tipo float.

Matrizes como argumentos de Funções

- Assim, a função media() recebe o endereço de uma variável float e não um valor.
- Devemos declarar esse endereço corretamente.
- O tipo float[] indica o endereço de uma variável float.
- O bom entendimento de endereços tornar-se-á imprescindível quando estudarmos ponteiros.

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

Matrizes São Passadas para Funções por Referência

- As matrizes são consideradas um tipo de dado bastante grande, pois são formadas por diversas variáveis.
- Por causa disso, em linguagem C, determina-se ser mais eficiente existir uma única cópia da matriz na memória, sendo, portanto, irrelevante o número de funções que a acessem.
- Assim, não são passados os valores contidos na matriz, somente o seu endereço de memória.

Matrizes São Passadas para Funções por Referência

- O que ocorre quando o endereço de uma matriz é passado para uma função como argumento?
- A função usa o endereço para acessar os elementos da matriz original, criada na função que chama.
- Isso significa que as alterações que a função efetuar nos elementos da matriz afetarão a matriz original.

Matrizes São Passadas para Funções por Referência

- Ao passar o nome de uma matriz para uma função não se cria uma nova cópia da matriz; a passagem é por referência.

Matrizes São Passadas para Funções por Referência

- No exemplo anterior, a função media() não altera os valores dos elementos da matriz.
- Vamos analisar um programa que ordena os valores de uma matriz e observar que a mudança da ordem é feita na matriz original, criada em main().

Matrizes São Passadas para Funções por Referência

Prog11.c

```c
/* OrdenaNum.C */
/* Ordena os valores da matriz */
#include <stdio.h>
#include <stdlib.h>
#define TAMANHO 50

void ordena(int[], int);/* protótipo */

int main()
{
    int Tab[TAMANHO], i, j;

    for(i=0; i<TAMANHO ; i++)
    {
        printf("Digite número ou zero para terminar: ");
        scanf("%d", &Tab[i]);

        if( !Tab[i] ) break;
    }

    ordena( Tab, i );
```

Matrizes São Passadas para Funções por Referência

```
23      for(j=0; j < i ; j++)
24          printf("%d\n", Tab[j]);
25      system("PAUSE");
26      return 0;
27  }
28
29  /* Ordena os valores da matriz */
30  void ordena( int lista[], int tamanho)
31  {
32      int i, j, temp;
33      for(i=0; i < tamanho - 1 ; i++)
34          for(j=i+1 ; j < tamanho ; j++)
35              if( lista[i] > lista[j] )
36              {
37                  temp = lista[j];
38                  lista[j] = lista[i];
39                  lista[i] = temp;
40              }
41  }
```

Roteiro

- Declaração de Matrizes
- Referenciação do elemento da matriz
- Matrizes de outros tipos de elementos
- Verificação de limites
- Inicialização de matrizes
- Matrizes de mais de uma dimensão
- Matrizes como argumentos de funções
- Matrizes são passadas para funções por referência
- A ordenação bolha

A Ordenação Bolha

- O processo de ordenação utilizado na função ordena() merece alguma explanação.
- A função começa considerando a primeira variável da matriz, lista[0].
- O objetivo é colocar o menor item da lista nessa variável.
- Assim, a função percorre cada um dos itens seguintes a fim de encontrar o menor deles.

A Ordenação Bolha

- Sempre que encontra um item menor, eles são trocados.
- Terminada a operação, segue-se ao próximo item: lista[1].
- Esse item deverá conter o próximo menor valor.
- Novamente, são realizadas as comparações e trocas.
- O processo continua até que a lista esteja ordenada.

A Ordenação Bolha

- O processo de ordenação é feito por meio de dois laços.
- O laço mais externo determina qual elemento da matriz será usado como base da comparação.
- O laço mais interno compara cada item seguinte com o de base e, quando encontra um item menor que o de base, faz a troca.
- A troca é feita no corpo do if por meio de uma variável temporária.

A Ordenação Bolha

- Esse método é chamado de ordenação bolha.
- Sua popularidade vem de sua simples utilização.
- No entanto, existem outros algoritmos, e alguns mais eficientes.
- A função main() imprime os valores dos elementos da matriz após o retorno da função ordena().
- Observe como a matriz foi efetivamente alterada.

Matrizes de Duas Dimensões como Argumento de Funções

- Para ilustrar o uso de uma matriz de duas dimensões passada como argumento para uma função, escreveremos um programa para avaliar a eficiência dos funcionários de uma loja quanto ao número de peças vendidas por cada um deles em três meses.
- O primeiro índice da matriz indica o número de funcionários da loja e o segundo, o número de meses a serem considerados.

Matrizes de Duas Dimensões como Argumento de Funções

Prog12.c

```c
/* Histograma.C */
/* Mostra matriz de duas dimensões como argumento de função */
#include <stdio.h>
#include <stdlib.h>
#define MES 3
#define FUNC 5

void histograma( int [][MES], int);/* protótipo */

int main()
{
    int pecas[FUNC][MES], i, j;

    for(i=0; i<FUNC ; i++)
        for(j=0; j<MES ; j++)
        {
            printf("Funcionário: %d\tmês %d: ", i+1, j+1);
            scanf("%d", &pecas[i][j]);
        }
    histograma(pecas,FUNC);
    system("PAUSE");
    return 0;
}
```

Matrizes de Duas Dimensões como Argumento de Funções

```
24  /* Imprime um histograma horizontal */
25  void histograma( int pecas[][MES], int nfunc)
26  {
27      const float MAXBARRA=50.0;
28      int max=0,temp=0, i, j, tam;
29
30      for(i=0; i<nfunc; i++)
31      {
32          for(j=0; j<MES ; j++)
33              temp += pecas[i][j];
34          if(max < temp)  max=temp;
35      }
36
37      temp=0;
38      for( i=0 ; i<nfunc; i++)
39      {
40          for( j=0; j<MES ; j++) temp += pecas[i][j];
41          printf("%2d - %5d:", i+1, temp);
42
43          tam = (int)((float)temp/(float)max*MAXBARRA);
44
45          for(j=0; j<tam; j++) printf("*");
46          printf("\n");
47      }
48  }
```

Matrizes de Duas Dimensões como Argumento de Funções

- O método de passagem do endereço da matriz para a função é idêntico ao da passagem de uma matriz de uma dimensão, não importando quantas dimensões tenha a matriz, visto que sempre passamos o seu endereço.

```
20          histograma(pecas,FUNC);
```

Matrizes de Duas Dimensões como Argumento de Funções

- Entretanto, a declaração da matriz na função e no seu protótipo é um tanto misteriosa:

```
25    void histograma( int pecas[][MES], int nfunc)
```

- Por que só estamos informando a segunda dimensão da matriz?
- Como não há verificação de limites, uma matriz com a primeira dimensão de qualquer valor pode ser passada para a função chamada.

Matrizes de Duas Dimensões como Argumento de Funções

- Mas em uma função que recebe uma matriz bidimensional deverá ser informada do comprimento da segunda dimensão para poder operar com declarações do tipo:

```
pecas[2][1];
```

- para encontrar a posição de memória em que pecas[2][1] está guardada, a função multiplica o primeiro índice (2) pelo número de elementos da segunda dimensão (MES) e adiciona o segundo índice (1), para finalmente somar o endereço da matriz (pecas).

Matrizes de Duas Dimensões como Argumento de Funções

- Caso o comprimento da segunda dimensão não seja conhecido, será impossível saber onde estão os valores.
- Nesse ponto, você já está apto a trabalhar com matrizes. Você sabe como declarar matrizes de diferentes dimensões e tipos, como inicializar matrizes, como referenciar um elemento particular de matriz e como passar uma matriz para uma função.

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;